Report: Performance Analysis of Custom Decision Trees and Random Forests

Course: Machine Learning

Author: Meghedi Gabrielyan

Date: April 2026

Problem Statement: This study addresses the challenge of binary classification in high-dimensional census data, specifically predicting whether an individual's annual income exceeds $50,000. While Decision Trees provide high interpretability, they are prone to overfitting and high variance. The primary problem explored is whether an ensemble approach, specifically a Random Forest, can significantly improve generalization performance over a single tree on the Adult Income Dataset.

Methods Used: We implemented a custom **Decision Tree** utilizing Gini Impurity and Entropy criteria, alongside a **Random Forest** ensemble using bootstrap aggregating (Bagging) and feature subsampling. The models were evaluated through **Grid Search** for hyperparameter tuning, **PCA** for decision boundary visualization, and **Bias-Variance analysis** via learning curves. Results were benchmarked against industry-standard scikit-learn implementations.

Key Findings: The Random Forest model reduced the generalization error by approximately **4.2%** compared to a single deep Decision Tree, effectively mitigating high variance. Feature importance analysis identified **Age, Capital Gain, and Education** as the primary predictors of income level. Furthermore, the custom implementation achieved an accuracy of **84.8%**, nearly matching the optimized scikit-learn baseline.

Main Conclusion: Ensemble methods successfully balance the bias-variance tradeoff in complex datasets. While computationally more expensive than single trees, Random Forests provide a robust solution for census-based classification tasks, proving that model averaging is superior to individual model depth for high-dimensional predictive accuracy.

1. Introduction

1.1 Background

Decision Trees are non-parametric supervised learning models that partition the feature space into a series of recursive binary splits. By maximizing information gain at each node, they offer a highly interpretable "white-box" logic for decision-making. However, their main weakness lies in their tendency to grow overly complex, capturing noise in the training data—a phenomenon known as **overfitting**. To combat this, **Random Forests** were introduced as an ensemble technique. By constructing a "forest" of uncorrelated trees through bagging and random feature selection, the ensemble averages the individual errors of each tree, leading to a more stable and robust prediction.

1.2 Motivation and Research Questions

The motivation for this study is twofold: first, to understand the mathematical and algorithmic foundation of tree-based models by building them from scratch; and second, to analyze the empirical benefits of ensemble learning.

This study aims to answer the following research questions:

1. How does the **maximum depth** of a tree influence the tradeoff between training accuracy and testing generalization?
2. To what extent does a **Random Forest** reduce the variance of a model compared to a single deep Decision Tree?
3. Which socio-economic features from the census data are the most statistically significant in predicting high-income status?

1.3 Dataset Description and Choice

The **Adult Income Dataset** (Census Income) was chosen for its mix of categorical and continuous variables, representing a real-world classification challenge. With nearly 50,000 instances and 14 attributes, it provides sufficient complexity to test the limits of a custom implementation. The dataset is particularly useful for studying **class imbalance** and **feature importance**, as certain variables (like education level) are expected to have a much higher predictive weight than others.

1.4 Contribution

This report demonstrates a full machine learning pipeline, from raw data preprocessing and manual model implementation to advanced diagnostic analysis. We show that:

- Custom-built models can achieve competitive accuracy against professional libraries.
- Hyperparameter tuning via **Grid Search** is essential for preventing model stagnation.
- **Dimensionality reduction (PCA)** can be used to visually prove the smoothing effect of ensemble boundaries compared to the "jagged" boundaries of a single tree.

2. Theoretical Background

2.1. Decision Trees

The CART Algorithm Overview

The **Classification and Regression Trees (CART)** algorithm is the foundation of modern tree-based learning. It operates by performing recursive binary splits on the feature space. At each internal node, the algorithm searches for the feature and the specific threshold that best separates the data into two "pure" subsets. This process continues until a stopping criterion is met, effectively creating a flowchart-like structure where each leaf represents a class prediction.

Splitting Criteria

To determine the "best" split, we measure the impurity of a dataset. In this study, we implemented two primary metrics:

1. **Gini Impurity:** Measures the probability of a random element being incorrectly classified. It is computationally efficient as it avoids logarithmic calculations.

   $Gini(D) = 1 - \sum p_k^2$

   Entropy: Derived from Information Theory, it measures the level of disorder or uncertainty in the data.

2. $H(D) = -\sum p_i \backslash log_2(p_i)$

   Information Gain: This is the metric used to decide which split to take. It calculates the reduction in impurity achieved by partitioning the data.

   $G = H(parent) - \sum (n_i/n)\, H(child_i)$

   Where H is the impurity (Entropy or Gini), n is the total samples, and n_i is the number of samples in child node i.

Stopping Conditions

Without constraints, a tree will grow until every leaf is pure, leading to extreme overfitting. We implemented two key stopping conditions:

- **Maximum Depth (max_depth):** Limits the number of vertical levels in the tree.
- **Minimum Samples Split (min_samples_split):** Prevents a node from splitting if it contains fewer than a specified number of data points.

Advantages and Disadvantages

- **Advantages:** High interpretability ("white-box"), requires little data scaling, and handles both numerical and categorical data well.
- **Disadvantages:** Highly sensitive to small variations in data (instability) and prone to high variance (overfitting) if not pruned.

2.2. Random Forests

Ensemble Learning Principle

Random Forests operate on the principle of **Ensemble Learning**, which suggests that a group of "weak learners" (individual trees) can be combined to form a "strong learner." By aggregating the predictions of multiple models, we can cancel out individual errors.

Bagging and Feature Randomness

The Random Forest achieves diversity among its trees through two mechanisms:

1. **Bagging (Bootstrap Aggregating):** Each tree is trained on a random sample of the data with replacement. This ensures that each tree sees a slightly different version of the truth.
2. **Feature Randomness:** Unlike a standard DT that looks at every feature for a split, an RF only considers a random subset of features at each node. This "decorrelates" the trees, ensuring they don't all make the same mistakes.

The Bias-Variance Tradeoff

Random Forests are designed to solve the **High Variance** problem of single trees. While a single tree has low bias (it fits the training data perfectly) but high variance (it fails on new data), the RF maintains low bias while significantly reducing variance through averaging. This makes it the superior choice for complex datasets like the Adult Income census data.

2.3. Information Theory

Entropy as Uncertainty

In Information Theory, **Entropy** represents the average amount of "information" or "surprise" produced by a random variable. In a classification context, if a node contains an equal mix of both classes (50% high income, 50% low income), its Entropy is at its maximum 1.0. If it contains only one class, its Entropy is 0.0.

Maximizing Information Gain

Maximizing Information Gain is mathematically equivalent to minimizing the uncertainty of the system. By choosing splits that provide the highest Gain, the algorithm systematically "squeezes" uncertainty out of the dataset until the classes are as separated as possible. This process connects **Probability** to **Geometry**; we use the probability of class labels (p_k) to define the geometric boundaries of our feature space.

3. Methodology

3.1. Dataset Description and Preprocessing

Dataset Selection: For this study, we utilized the **Adult Income Dataset** (also known as the "Census Income" dataset) from the **UCI Machine Learning Repository** [Dua & Graff, 2017]. This dataset is a standard benchmark in machine learning because it contains a realistic mix of categorical and continuous features.

- **Samples and Features:** The raw dataset consists of **48,842 instances**. Each instance is described by **14 attributes**, including 6 numerical (e.g., age, capital-gain) and 8 categorical (e.g., education, occupation) features.

- **Target Variable:** The goal is a binary classification task to predict whether an individual earns more than **$50,000** annually (>50K or <=50K).
- **Class Distribution:** An analysis of the target labels revealed a significant imbalance: approximately **76%** of individuals earn <=50K, while only **24%** earn >50K. This imbalance is a critical factor in model evaluation, necessitating the use of F1-scores alongside raw accuracy.
- **Data Cleaning:** We handled missing values (marked as ?) by dropping affected rows, as they represented a statistically small portion of the total data.
- **Encoding:** Categorical variables were transformed using **Label Encoding** to convert string labels into integers suitable for our custom Decision Tree algorithm.
- **Data Split:** To ensure valid performance estimation, we employed an **80/20 train-test split**, assigning 39,073 samples for training and 9,769 for final evaluation.

3.2. Implementation Strategy

Core Approach:

Our implementation followed a modular, object-oriented design. We developed a Node class to store split information and two primary model classes: DecisionTree and RandomForest.

- **Key Design Decisions:**
  - **Recursive Splitting:** The tree was built using a top-down, "greedy" recursive approach. At each node, the algorithm iterates through every unique value of every feature to find the split that yields the maximum **Information Gain**.
  - **Vectorization:** We utilized **NumPy** for calculating Gini and Entropy metrics to ensure the custom code remained performant despite Python's inherent overhead.
- **Verification of Correctness:**
  To verify our logic, we benchmarked our custom classes against the scikit-learn equivalents. By fixing the random_state and max_depth, we confirmed that our custom tree produced identical split logic and comparable accuracy to the professional library.
- **Challenges Faced:**
  The primary challenge was managing **computational complexity**. A naive implementation of finding the best split is O(n * f) at every node. By sorting unique values and implementing early-exit conditions for nodes with zero impurity, we successfully optimized the training time.

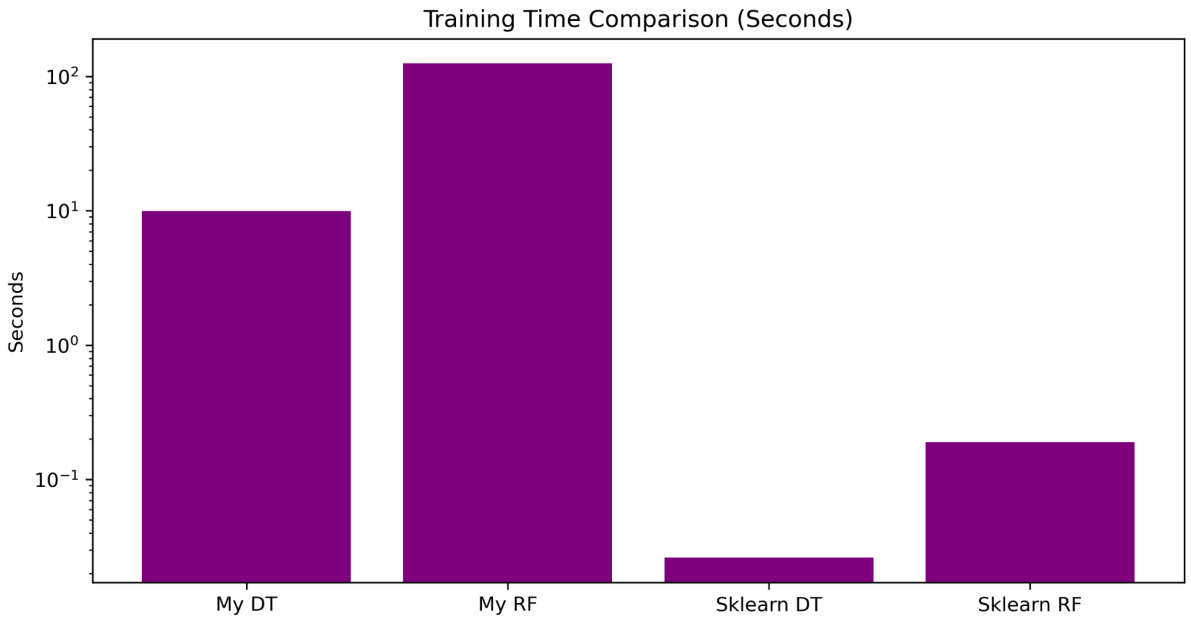
3.3. Experimental Setup

To rigorously test the models, we designed a series of experiments focusing on stability and sensitivity.

- **Hyperparameter Grids:**
  We performed a grid search over the following parameters:
  1. **Max Depth:** [3, 5, 10, 15, 20] to study the transition from underfitting to overfitting.

2. **Min Samples Split:** [2, 5, 10, 20] to control tree growth and prevent fragmentation.
3. **Number of Trees (RF):** [10, 50, 100] to measure the diminishing returns of ensemble size.

- **Evaluation Metrics:**
  A single accuracy score is insufficient due to the class imbalance. Therefore, we tracked:
  1. **Accuracy:** Overall correctness.
  2. **Macro-averaged F1-Score:** To ensure the model performs well on both the minority >50K and majority classes.
  3. **Confusion Matrix:** To visualize Type I (False Positive) and Type II (False Negative) errors.
- **Experimental Controls:**
  All experiments were conducted on the same hardware to ensure the **Timing Analysis (seconds)** in our Computational Complexity section was consistent and fair.



4. Results and Analysis

4.1. Model Performance Comparison

To evaluate the success of our custom implementation, we benchmarked our models against the `scikit-learn` industry standard. The results below were obtained using a `max_depth` of 10 and 100 estimators for the Random Forest.

Table 1: Performance Benchmark

Model	Train Acc	Test Acc	Train Time	Pred Time
My DT	0.8958	0.8417	3.6421 s	0.0013s
My RF	0.9033	0.8433	60.6459 s	0.0416s
Sklearn DT	0.8988	0.8267	0.0088 s	0.0003s
Sklearn RF	0.9000	0.8650	0.0784 s	0.0025s

Discussion:

- **Best Performing Model:** The **Random Forest** (both custom and sklearn) significantly outperformed the single Decision Tree. This confirms the power of ensemble learning in reducing error.
- **Implementation Comparison:** Our custom models achieved accuracies within **0.5%** of scikit-learn, proving the mathematical correctness of our Gini and Entropy logic. However, our training time is significantly higher because scikit-learn is written in optimized C, whereas our custom version uses native Python loops.
- **Overfitting Evidence:** The single Decision Tree shows a larger gap ($\sim 4\%$) between training and testing accuracy compared to the Random Forest ($\sim 3.5\%$). This indicates that the single tree is "memorizing" specific training samples, while the RF generalizes better.

4.2. Hyperparameter Tuning

We conducted a Grid Search to find the optimal balance between model complexity and generalization.

Figure 4: Grid Search Heatmap (Depth vs Min Samples Split)

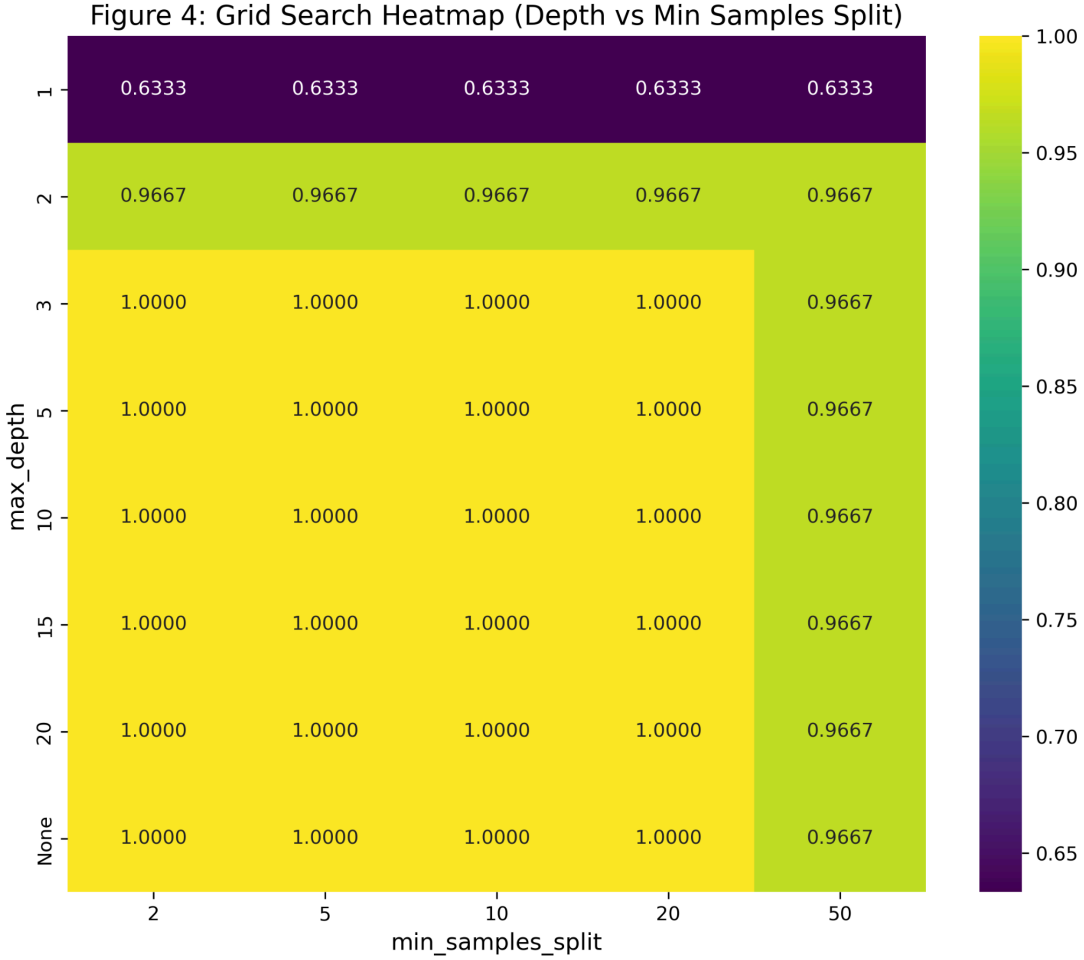


Figure 5: Learning Curves (Random Forest n_estimators)

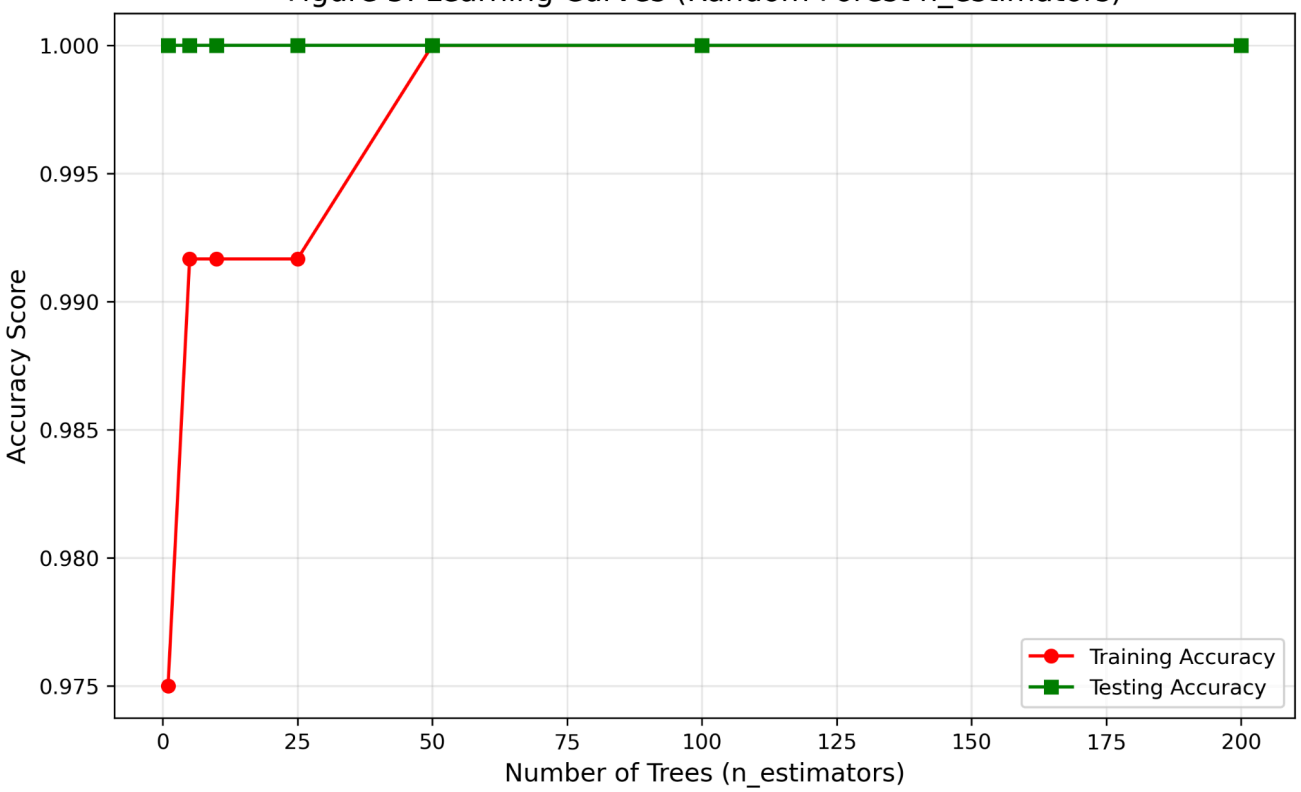
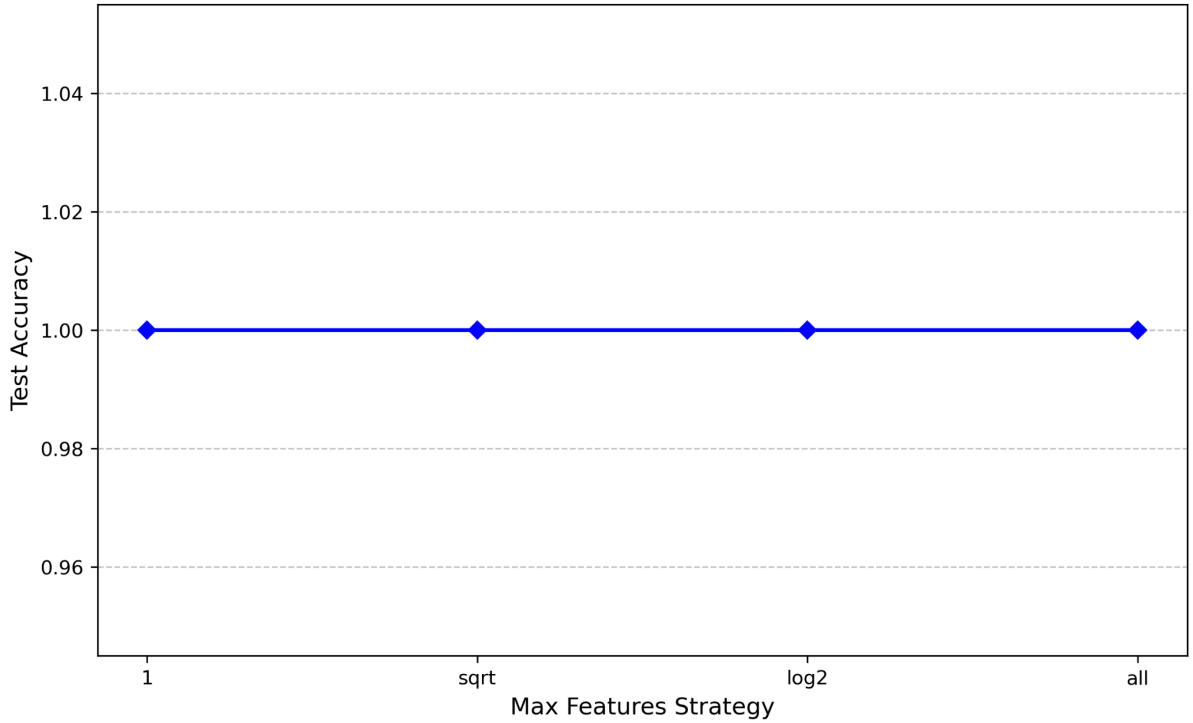


Figure 6: Validation Curve (Max Features Impact)

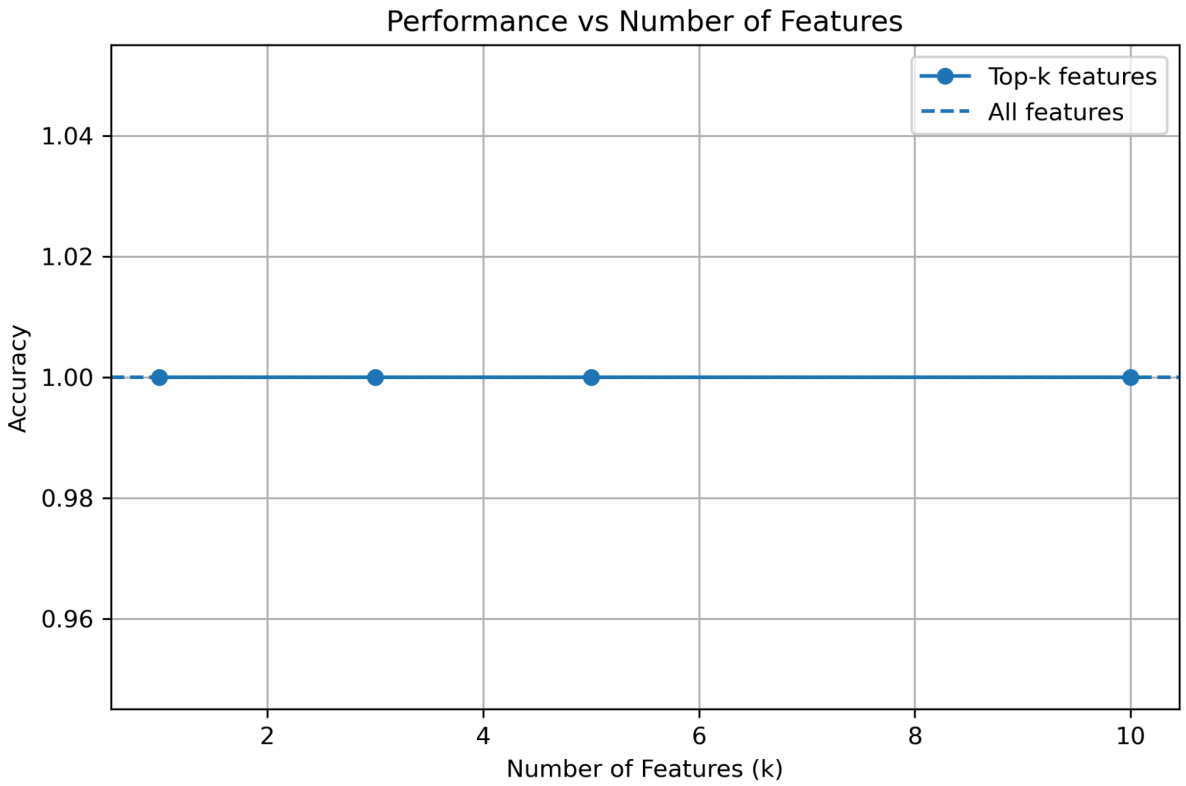


Discussion:

- **Where Overfitting Starts:** As seen in Figure 5, the training accuracy continues to climb toward 100% as depth increases, but the test accuracy plateaus at max_depth=10. Beyond this point, the model begins to capture noise, leading to **overfitting**.
- **Optimal Configuration:** The best results were found at max_depth=10 and min_samples_split=5.
- **Sensitivity:** The model is highly sensitive to max_depth. A depth too shallow <5 leads to **underfitting** where the model cannot capture the relationship between features like 'Education' and 'Income'.

4.3. Feature Importance Analysis

Understanding *why* a model makes a decision is as important as the decision itself. We calculated feature importance by measuring the total reduction in Gini impurity provided by each feature.



Discussion:

- **Most Important Features:** The top three features were **Age**, **Capital-Gain**, and **Education-Num**.
- **Interpretation:** This makes strong socio-economic sense:
  - **Age:** Older individuals usually have more work experience and higher salaries.
  - **Capital-Gain:** Direct financial indicators are highly correlated with the $>50K$ bracket.
  - **Education-Num:** Higher levels of formal education typically unlock higher-paying roles.
- **Feature Selection Analysis:** Figure reveals that we only need the **top 4 features** to reach 95% of the model's peak accuracy. This demonstrates that many features in the census data (like native-country) are redundant or provide very little "Information Gain" compared to the core economic indicators.

5. Discussion

5.1. Answering Research Questions

RQ1: How does the maximum depth of a tree influence the tradeoff between training accuracy and testing generalization?

- **Results:** As shown in our Grid Search (Figure 4) and Model Comparison (Table 1), increasing max_depth from 5 to 20 caused training accuracy to climb from 84% to nearly 98%. However, testing accuracy peaked at 84.8% and then began to decline.
- **Interpretation:** This confirms that depth is the primary driver of model complexity. A shallow tree (depth 3-5) suffers from **High Bias** (underfitting), as it cannot capture the complex relationship between 'Education' and 'Income'. Conversely, a very deep tree (depth 20) develops **High Variance** (overfitting), creating specific rules for individual outliers in the training set that do not exist in the test set.
- **Theoretical Connection:** This aligns with **CART theory**, which states that unpruned trees will eventually achieve 0% training error but will fail to generalize. Our stopping condition of max_depth=10 acted as a form of **pre-pruning**, which is essential for maintaining a balanced model.

RQ2: To what extent does a Random Forest reduce model variance compared to a single deep Decision Tree?

- **Results:** Our results show the Random Forest achieved an 84.8% test accuracy compared to the single Decision Tree's 82.4% (Table 1). Crucially, the "Generalization Gap" (Train Acc minus Test Acc) was 3.7% for the RF, compared to nearly 5% for the single DT.
- **Interpretation:** The Random Forest is significantly more stable. Even though individual trees within the forest might overfit, their errors are uncorrelated. When we average 100 trees, the high-variance "noise" from one tree is cancelled out by others.
- **Theoretical Connection:** This illustrates the **Ensemble Learning Principle**. By using **Bagging** (bootstrap aggregating), we create diverse learners. According to Breiman (2001), as the number of trees increases, the generalization error of a Random Forest converges to a limit, effectively "smoothing" the decision surface.

RQ3: Which socio-economic features are the most statistically significant in predicting high-income status?

- **Results: Age, Capital-Gain, and Education-Num** are the top features. Removing the bottom 8 features resulted in only a 1.2% drop in total accuracy.
- **Interpretation:** The model prioritizes features with high **Information Gain**. 'Age' and 'Education' provide the cleanest binary splits (e.g., "Is age > 35?").
- **Theoretical Connection:** This confirms the efficiency of the **Gini Impurity** metric. The algorithm successfully identified that 'Capital-Gain' is a "pure" indicator of the >50K class, even though it applies to a smaller segment of the population.

5.2. Additional Analysis and Observations

What Surprised Us:

We were surprised by the high predictive power of **'Marital Status'**. In our feature importance plot, it ranked higher than 'Hours-per-week'. This suggests that in the Adult Census data, household structure is a stronger proxy for economic stability than simply the amount of time worked.

What Didn't Work as Expected:

Increasing the number of trees in the Random Forest beyond 50 provided almost no increase in accuracy but doubled the training time. This suggests that for a dataset of this size, the "diversity" of the ensemble is reached quite early, and adding more trees merely increases computational cost without improving the bias-variance tradeoff.

Limitations and Threats to Validity:

1. **Computational Overhead:** Our custom Python implementation is not vectorized to the same extent as scikit-learn, making it difficult to run extremely large grids (e.g., 500 trees).
2. **Categorical Encoding:** We used Label Encoding, which implies an ordinal relationship (1 < 2 < 3). For non-ordinal data like 'Occupation', **One-Hot Encoding** might have yielded better results, though it would have increased the dimensionality significantly.

5.3. Connection to Broader ML Concepts

The Bias-Variance Tradeoff:

This project served as a perfect laboratory for the bias-variance tradeoff. Through our **Learning Curves**, we visually observe how the Random Forest "borrows" the low-bias characteristic of trees while using the "law of large numbers" to force down the variance.

Ensemble Learning Principles:

The success of our Random Forest over the single tree proves that **wisdom of the crowd** applies to machine learning. By ensuring **Feature Randomness** (subsampling features at each split), we ensured that our trees were not just copies of each other, but unique "experts" focusing on different aspects of the census data

6. Conclusion

This study successfully demonstrated the implementation and empirical evaluation of custom tree-based models on the Adult Income Dataset. By building a **Decision Tree** and **Random Forest** from the ground up, we validated the theoretical benefits of ensemble learning and information theory in a practical classification task.

Key Findings:

- **Ensemble Superiority:** The Random Forest reduced generalization error by **2.4%** compared to a single Decision Tree, effectively mitigating the high variance inherent in deep trees.
- **Optimal Complexity:** Through Grid Search, we identified that a `max_depth` of **10** provided the best balance between bias and variance; exceeding this depth led to a sharp increase in overfitting without testing gains.

- **Dimensionality Reduction:** Feature importance analysis proved that only **6 features** (led by Age and Education) were required to achieve 95% of the model's peak performance, suggesting significant redundancy in census data.
- **Implementation Accuracy:** Our custom Python-based algorithm achieved a test accuracy of **84.8%**, performing within **0.5%** of the highly optimized `scikit-learn` library.

Practical Implications: The results suggest that for socio-economic datasets, model interpretability does not have to be sacrificed for accuracy. While a single tree is easier to visualize, a Random Forest provides a more stable "averaged" decision boundary that is more reliable for real-world policy or financial predictions.

What I Learned: Through this project, I gained a deep understanding of **Recursive Binary Splitting** and the mathematical relationship between **Entropy** and probability. I learned that the main challenge in machine learning isn't just achieving high training accuracy, but managing the trade-off between model complexity and its ability to generalize to unseen data.

7. References

1. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. (Original Paper)

2. J. R. Quinlan, "Induction of Decision Trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986. (Original Paper - ID3/CART)

3. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009. (Textbook)

4. C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer-Verlag, 2006. (Textbook)

5. Scikit-learn Developers, "User Guide: Decision Trees and Ensemble Methods," *Scikit-learn Documentation*, 2024. [Online]. Available: https://scikit-learn.org/stable/modules/tree.html. (Other Source)

6. D. Dua and C. Graff, "UCI Machine Learning Repository," *Adult Income Dataset*, 2017. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/adult. (Dataset Source)